Encoder–decoder cho dữ liệu chuỗi

Seq2seq, attention và Transformer

Tài liệu lý thuyết gắn với notebook của dự án

08/10/2026

Tóm tắt

Báo cáo khảo sát cách một mạng encoder–decoder biến chuỗi nguồn thành chuỗi đích có độ dài khác. Tài liệu đi từ RNN seq2seq dùng một vector ngữ cảnh đến attention và Transformer, đồng thời giải thích dịch phải nhãn đích, teacher forcing, mask, hàm mất mát và suy luận tự hồi quy. Ví dụ câu dịch chỉ dùng để theo dõi luồng dữ liệu; dự án hiện chưa huấn luyện một mô hình dịch máy.

Mục lục

1 Định nghĩa bài toán **2**

2 RNN encoder–decoder cơ bản **2**

2.1 Encoder đọc lần lượt chuỗi nguồn 2

2.2 Decoder sinh token 2

2.3 Ví dụ dịch phải chuỗi đích 2

3 Loss, lan truyền ngược và suy luận **3**

4 Attention: truy cập nhiều trạng thái nguồn **3**

5 Transformer encoder–decoder **4**

6 Thiết kế một thí nghiệm seq2seq có thể đánh giá **4**

7 Kết luận **5**

1 Định nghĩa bài toán

Cho chuỗi nguồn $x = (x_1, \ldots, x_T)$ và chuỗi đích $y = (y_1, \ldots, y_U)$, ta muốn ước lượng phân phối có điều kiện

$$P(y \mid x) = \prod_{t=1}^{U+1} P(y_t \mid y_{<t}, x), \tag{1}$$

trong đó $y_{U+1} = \langle \text{EOS} \rangle$ đánh dấu kết thúc. Độ dài T và U không cần bằng nhau. Ví dụ nhiệm vụ gồm dịch máy, tóm tắt và chuẩn hóa văn bản. Encoder đọc nguồn; decoder tạo phân phối cho mỗi token đích.

Đầu vào văn bản phải được tách thành token theo một quy tắc nhất quán và ánh xạ thành ID. $\langle \text{BOS} \rangle$ khởi động decoder, $\langle \text{EOS} \rangle$ báo dừng, $\langle \text{PAD} \rangle$ làm các chuỗi trong batch có cùng độ dài. PAD không phải một từ cần dự đoán; vị trí PAD phải được loại khỏi loss, và khi cần khỏi attention.

2 RNN encoder–decoder cơ bản

2.1 Encoder đọc lần lượt chuỗi nguồn

Gọi $E_s[x_t] \in \mathbb{R}^d$ là embedding của token thứ t. RNN đơn giản tính

$$h_t = \tanh\big(E_s[x_t]W_x + h_{t-1}W_h + b_h\big), \quad h_0 = 0. \tag{2}$$

Trạng thái cuối $c = h_T$ tóm lược nguồn. Encoder được dùng lặp lại theo thời gian với cùng bộ trọng số; không phải mỗi vị trí một mạng riêng. Mô hình LSTM seq2seq của Sutskever, Vinyals và Le dùng cơ chế trạng thái khác RNN tanh đơn giản, nhưng vẫn theo nguyên tắc mã hóa chuỗi rồi giải mã chuỗi [1].

2.2 Decoder sinh token

Một dạng RNN decoder minh họa là

$$s_0 = c, \tag{3}$$

$$s_t = \tanh\big(E_t[\tilde{y}_{t-1}]U_x + s_{t-1}U_h + b_s\big), \tag{4}$$

$$P(y_t = j \mid y_{<t}, x) = \text{softmax}(s_t W_o + b_o)_j. \tag{5}$$

$\tilde{y}_0 = \langle \text{BOS} \rangle$. Trong huấn luyện teacher forcing, $\tilde{y}_{t-1}$ là token đúng ở bước trước; khi suy luận, nó là token do mô hình vừa chọn. LSTM còn có cell state, nên không thể mô tả đầy đủ LSTM chỉ bằng một phương trình hidden state như trên.

2.3 Ví dụ dịch phải chuỗi đích

Xét nguồn “tôi yêu mèo” và đích minh họa “i love cats”. Bảng sau là cặp đầu vào–nhãn để giải thích, *không phải đầu ra từ mô hình đã huấn luyện*. Nếu đưa chính token y_t vào decoder ở bước cần dự đoán y_t, mô hình nhìn thấy đáp án và phép đánh giá bị rò rỉ.

Bước	Decoder input khi train	Nhãn cần đoán	Input khi suy luận
1	⟨BOS⟩	i	⟨BOS⟩
2	i thật	love	Token dự đoán ở bước 1
3	love thật	cats	Token dự đoán ở bước 2
4	cats thật	⟨EOS⟩	Token dự đoán ở bước 3

Bảng 1: Chuỗi đầu vào decoder được dịch phải một vị trí so với nhãn.

3 Loss, lan truyền ngược và suy luận

Với N chuỗi đích, đặt $m_{i,t} = 1$ nếu token thứ t của mẫu i hợp lệ, ngược lại bằng 0 nếu là PAD. Tổng số token hợp lệ $M = \sum_{i,t} m_{i,t}$. Cross-entropy chuẩn hóa theo số token là

$$\mathcal{L} = -\frac{1}{M} \sum_{i,t} m_{i,t} \log P_{\theta,\phi}(y^*_{i,t} \mid y^*_{i,<t}, x_i). \tag{6}$$

Lan truyền ngược qua thời gian (backpropagation through time) cộng ảnh hưởng của một tham số RNN ở mọi bước mà nó được dùng. Gradient của loss từ decoder có thể đi về trạng thái c và tiếp tục cập nhật encoder. Khi chuỗi dài, đường truyền này khó tối ưu; LSTM/GRU và attention là những hướng xử lý, nhưng không bảo đảm loại bỏ mọi lỗi.

Khi suy luận, mô hình không biết nhãn thật. Greedy decoding chọn $\arg\max_j P(y_t = j \mid y_{<t}, x)$ tại từng bước và dừng khi gặp EOS hoặc đạt độ dài tối đa. Beam search giữ nhiều tiền tố ứng viên và so sánh điểm tích lũy, nhưng cũng chỉ là tìm kiếm xấp xỉ và cần quy tắc xử lý độ dài. Teacher forcing tạo sự khác biệt phân phối đầu vào giữa train và suy luận: một lỗi sớm có thể ảnh hưởng nhiều token về sau.

4 Attention: truy cập nhiều trạng thái nguồn

Nén cả chuỗi vào duy nhất h_T tạo nút thắt thông tin, nhất là với nguồn dài. Attention cho decoder nhìn tập $H = (h_1, \ldots, h_T)$ và tạo ngữ cảnh riêng tại mỗi bước [2]:

$$e_{t,i} = \text{score}(s_{t-1}, h_i), \tag{7}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j \in \mathcal{V}_x} \exp(e_{t,j})}, \quad i \in \mathcal{V}_x, \tag{8}$$

$$c_t = \sum_{i \in \mathcal{V}_x} \alpha_{t,i} h_i. \tag{9}$$

$\mathcal{V}_x$ là các vị trí nguồn thật; vị trí PAD bị mask. Các trọng số $\alpha_{t,i}$ không âm và có tổng bằng 1 trên vị trí hợp lệ. Decoder có thể dùng c_t khi cập nhật trạng thái hoặc khi tạo logits. Attention học theo mục tiêu dự đoán token; trọng số của nó không mặc nhiên là lời giải thích đầy đủ về quyết định của mô hình.

5 Transformer encoder–decoder

Transformer gốc thay các phép cập nhật RNN tuần tự trong từng tầng bằng attention và mạng feed-forward [3]. Mỗi token được biểu diễn bằng embedding cộng thông tin vị trí. Với $Q = XW_Q$, $K = XW_K$, $V = XW_V$ và chiều khóa d_k:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V. \tag{10}$$

Mask M có giá trị 0 cho cặp vị trí được phép và giá trị rất âm cho cặp bị chặn. Multi-head attention tính nhiều phép chiếu Q, K, V rồi ghép kết quả. Khối Transformer còn có feed-forward, residual connection và chuẩn hóa; công thức attention đơn lẻ chưa mô tả toàn mạng.

Khối	Q lấy từ	K, V lấy từ và mask
Encoder self-attention	Biểu diễn nguồn	Biểu diễn nguồn; chặn PAD
Decoder masked self-attention	Tiền tố đích	Tiền tố đích; chặn PAD và vị trí tương lai
Decoder cross-attention	Biểu diễn decoder	Đầu ra encoder; chặn PAD nguồn

Bảng 2: Ba nơi dùng attention trong Transformer encoder–decoder.

Ở thời điểm train, causal mask cho phép tính các vị trí đích song song trong một lượt truyền xuôi mà không nhìn token tương lai. Khi sinh tự hồi quy, token tiếp theo vẫn phụ thuộc các token vừa được tạo nên quá trình tạo chuỗi diễn ra tuần tự. BERT thường là encoder-only; GPT thường là decoder-only; T5/BART là ví dụ encoder–decoder. Các tên gọi này chỉ kiến trúc tổng quát, không thay cho mô tả nhiệm vụ và cách huấn luyện cụ thể.

6 Thiết kế một thí nghiệm seq2seq có thể đánh giá

Notebook của dự án mới minh họa bằng lời và công thức, *chưa cài hoặc chạy mô hình dịch*. Để triển khai thí nghiệm thật, cần xác định trước:

1. **Dữ liệu:** cặp nguồn–đích, nguồn gốc, quyền sử dụng, bộ tách token, cách xử lý từ ngoài từ vựng và giới hạn độ dài.
2. **Phép chia:** train/validation/test tách biệt; loại cặp trùng hoặc gần trùng giữa các tập. Chỉ fit từ vựng và chọn hyperparameter trên train/validation.
3. **Đối chứng:** ví dụ RNN encoder–decoder cơ bản so với cùng cấu hình thêm attention; nếu dùng Transformer, báo rõ số tầng, kích thước và ngân sách huấn luyện.
4. **Đo lường:** token cross-entropy trên test, exact match cho tác vụ có đáp án xác định, và đánh giá chất lượng văn bản phù hợp nhiệm vụ. Báo cả ví dụ đúng/sai, độ dài và trường hợp EOS xuất hiện quá sớm.
5. **Ablation:** thay đổi một yếu tố mỗi lần, chẳng hạn có/không attention hoặc độ dài nguồn, trong khi giữ phép chia và điều kiện đánh giá cố định.

Không được dùng câu dịch minh họa ở Bảng 1 như điểm số của một hệ thống. Test chỉ dùng sau khi chốt checkpoint và cách giải mã theo validation.

7 Kết luận

Seq2seq học phân phối token đích có điều kiện theo chuỗi nguồn và tiền tố đích. RNN cơ bản chuyển nguồn qua trạng thái cuối, attention cho phép tham chiếu nhiều vị trí nguồn, còn Transformer tổ chức các phép tham chiếu bằng self-attention và cross-attention có mask. Dù kiến trúc nào, việc dịch phải đầu vào decoder, loại PAD khỏi loss và tách train khỏi suy luận là điều kiện để thí nghiệm có nghĩa.

Tài liệu tham khảo

[1] I. Sutskever, O. Vinyals, Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, 2014. https://arxiv.org/abs/1409.3215.

[2] D. Bahdanau, K. Cho, Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. https://arxiv.org/abs/1409.0473.

[3] A. Vaswani et al., *Attention Is All You Need*, 2017. https://arxiv.org/abs/1706.03762.

[4] Trương Xuân Nam, *Xử lý ngôn ngữ tự nhiên*, `slide/NLP-1.pdf`, trang 10 và 16.

[5] `encoder_decoder_minh_hoa.ipynb`, phần 1.2–1.3: minh họa seq2seq và attention.